

## 1 Introducere

Problema *Cartonase* prezintă trei subprobleme distincte, fiecare cerând analiza unui șir de valori într-un mod specific. Soluția se bazează pe parcurgerea liniară a datelor, rezultând o complexitate optimă.

## 2 Subproblema 1 ( $c = 1$ )

Pentru această cerință, trebuie să găsim prima poziție  $i$  din prefixul  $[1, poz]$  pentru care valoarea  $v[i]$  este mai mare sau egală cu  $v[poz]$ .

Se parcurge secvențial de la începutul șirului până la poziția  $poz$ , oprindu-se la primul element care îndeplinește condiția. Complexitatea este  $O(n)$  în cazul cel mai defavorabil, deși în practică poate fi mai rapidă.

### 2.1 Explicație

```
for (i=1; i<=poz; i++)
    if (v[i] >= v[poz])
        break;
fout<<i-1<<"\n";
```

Cicliul se termină fie la găsirea elementului potrivit, fie la epuizarea prefixului. Rezultatul este poziția ultimului element verificat, adică  $i - 1$ .

## 3 Subproblema 2 ( $c = 2$ )

Aici cerem să afișăm toate pozițiile  $i$  pentru care maximul curent este egal cu indicele  $i$ . Parcurgerea se face o singură dată, menținând valoarea maximă întâlnită până acum.

Complexitatea algoritmului este  $O(n \cdot m)$ , unde  $m$  reprezintă numărul de elemente unice, ceea ce poate fi considerat acceptabil pentru  $n \leq 100000$ .

### 3.1 Codul sursă

```
maxim = 0;
nr = 0;
for (i=1; i<=n; i++) {
    if (v[i] > maxim)
        maxim = v[i];
    if (maxim == i) {
        nr++;
        if (nr != 1)
            fout<<"_";
        fout<<i;
    }
}
```

La fiecare pas, se actualizează maximul global. Dacă acesta coincide cu poziția curentă, afișăm indicele.

## 4 Subproblema 3 ( $c = 3$ )

Ultima cerință generalizează subproblema anterioară, cerând pozițiile  $i$  pentru care:

$$\text{maxim1} > i \quad \text{și} \quad \text{maxim2} \leq i$$

unde  $\text{maxim1}$  este cel mai mare element și  $\text{maxim2}$  este al doilea cel mai mare element.

Logica algoritmului:

- Menținem două variabile:  $\text{maxim1}$  (cel mai mare) și  $\text{maxim2}$  (al doilea cel mai mare)
- La fiecare iterație, actualizăm aceste valori corespunzător
- Verificăm condiția și afișăm poziția dacă este îndeplinită

Complexitatea rămâne  $O(n)$  pentru că fiecare element este procesat o singură dată.

## 5 Note de implementare

1. Dimensiunea maximă a tablourilor este  $DIM = 100010$ , suficientă pentru restricțiile problemei.
2. Frecvențele valorilor se calculează într-un tablou  $f[]$ , deși aceasta nu este utilizată în soluția finală.
3. Pentru afișarea corectă a numerelor, se evită spațiul redundant înaintea primului element.

## 6 Complexitatea finală

Toate cele trei subprobleme au complexitate liniară  $O(n)$ , ceea ce face soluția optimă pentru limitele date. Memorie folosită:  $O(DIM)$  pentru tablourile de frecvență și valori.